

AuraGuard Architecture & Implementation Guide

FS Dotnet Internship Project Deliverables Reference Sheet

Project Identification

Project ID	Domain	Tech Track & Features
FSD28-INTERN-133	Fintech / Insurtech	Full Stack .NET (React + ASP.NET Core API + MySQL)

Project Idea & Concept

AuraGuard is a digital-first, paperless insurance administration platform designed to eliminate traditional operational bottlenecks. By leveraging automated rules execution and digital-first coordination pipelines, AuraGuard digitizes the entire lifecycle of policy management, transaction billing, compliance vault checking, and claims processing.

Key Modules

- **Dynamic Policy Catalog & Onboarding:** Fetches and displays current coverage options, max coverage limits, and premium rates from the database, enforcing a strict age-verification gate (18+) via DOB calculation.
- **Centralized Compliance Files Vault:** Encrypted file cabinet mapping Aadhaar cards, medical documents, and accident reports to specific policies, tracking the name and role of the uploader.
- **Split Claim Settlement Processing:** Direct routing of approved agent claims into a pending settlement queue, with final disbursement release restricted to admin verification.
- **Chatbot Virtual Assistant ("Aura"):** Smart natural language chatbot assisting with claim status updates using explicit Policy ID parses.
- **Immutable Agent Activity Logs:** Immutable trace ledger tracking critical agent operations (approvals, document verification, premium collections) for admin auditing.

Architecture Components

- **React Frontend:** Main dashboard rendering visual AreaChart collections telemetry, notifications panels, document uploaders, chat comments, and user profiles. Uses customized CSS design tokens for slate/indigo light-theme accessibility.
- **ASP.NET Core REST API:** Web API back-end built in C# using Entity Framework Core, executing Gregorian age checks, JWT token authorizations, and private comment logs filters.
- **MySQL Database:** Stores primary structures for Users, Customers, Policies, Payments, Claims, and Documents, maintaining cascading deletes configuration rules.
- **AI Chatbot Service:** Custom regex-based natural language parser inside the React client extracting Policy IDs and query targets.

AI & Rule Engine Details

- **Automated Claims Validation:** Executes code checks validating claim amounts against policy limits, verifying policy status (active vs expired), and checking accident dates against coverage windows.
- **Aura Query Parsing:** Regex-based pattern matching mapping user queries like 'policy 1', 'policy #2', or 'policy

AuraGuard Architecture & Implementation Guide

FS Dotnet Internship Project Deliverables Reference Sheet

id 3' directly to their database indexes.

Recommended Libraries & Tools

- **Lucide React:** Vector icon package for dashboard navigation.
- **Recharts:** Responsive charting library for financial telemetry.
- **Entity Framework Core:** Object-relational mapper for database queries.
- **BCrypt.Net:** Secure hashing library for password storage.

Implementation Flow (Step-by-Step)

- 1. Onboarding:** Customer inputs their DOB; frontend and backend validate that the age is 18+ before saving the profile.
- 2. Policy Vaulting:** Customer uploads compliance documents; the system records file paths and logs the uploader name and role.
- 3. Claim Filing:** Customer files a claim; the backend rules engine automatically checks validity, amount boundaries, and coverage dates.
- 4. Agent Audit:** Agent reviews claims and uploads; operations like approvals or document rejections are written to the audit log ledger.
- 5. Split Approval:** Agent approves the claim, routing it to Pending Settlement. The Admin reviews the transaction and settles it.
- 6. Chat Tracking:** Customer queries Aura; the chatbot parses the message using Policy IDs, retrieves DB claim records, and displays active statuses.